

---

**LAPORAN PRAKTIKUM**  
**MATA KULIAH TEKNOLOGI IOT II**

---

**MODUL 4: KOMUNIKASI ASINKRON DAN WEBSOCKETS**



**Oleh:**

**Nama: Aila Nadianti**

**NPM : 25782059**

**Kelas : 3C**

**Dosen:**

**Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.**

**Agus Ambarwari, S.Pd., M.Kom.**

**TEKNOLOGI REKAYASA INTERNET**  
**JURUSAN TEKNOLOGI INFORMASI**  
**POLITEKNIK NEGERI LAMPUNG**

**2026**

## 4.3 Teori Dasar

### 4.3.1 Keterbatasan HTTP Standar

Seperti yang telah Anda buktikan pada Modul 3, *Web Server* HTTP bersifat **Sinkron (Request-Response)**. Jika NodeMCU sedang memproses suatu perintah, ia tidak dapat melayani hal lain. Selain itu, untuk memperbarui data sensor (seperti suhu), *browser* harus memuat ulang seluruh kerangka halaman web menggunakan *Meta Refresh* atau tombol manual. Proses ini sangat boros *bandwidth* dan menyebabkan transisi layar yang kasar (*flickering*).

### 4.3.2 *Asynchronous Web Server* dan *WebSockets*

**WebSockets** adalah protokol komunikasi yang memecahkan masalah HTTP dengan membuka koneksi pipa data yang bersifat persisten (terus terhubung) dan dua arah (*full-duplex*) antara *Client* (browser) dan *Server* (NodeMCU).

- **Analogi Sederhana:** HTTP bekerja seperti "mengirim pesan surat" (Klien mengirim surat permintaan, Server membalas, lalu koneksi terputus). Sedangkan WebSockets bekerja layaknya "panggilan telepon" (koneksi tetap terbuka secara konstan, dan Klien maupun Server bisa saling berbicara kapan saja secara bersamaan).
- **Keunggulan Arsitektur (*Push Data*):** Karena jalur komunikasi selalu siaga, Server NodeMCU tidak perlu lagi menunggu ditanya. Begitu suhu berubah atau *push button* fisik ditekan, NodeMCU dapat secara proaktif **mendorong (*push*)** paket data kecil (berformat JSON) ke *browser*. Melalui JavaScript, *browser* akan langsung memperbarui tampilan angkanya saja, tanpa memuat ulang (*refresh*) kerangka halaman web HTML secara keseluruhan.
- Pustaka eksternal yang paling tangguh dan menjadi standar industri di ekosistem ESP8266 untuk menangani protokol ini adalah ESPAsyncWebServer.

## 4.4 Persiapan Praktikum

### 4.4.1 Kebutuhan Perangkat Keras

Pastikan komponen berikut tersedia di meja Anda:

- *Board* NodeMCU ESP8266 (1 buah)
- Kabel Micro USB (1 buah)
- *Breadboard* (1 buah)
- LED 5mm (1 buah, atau 2 buah jika mengerjakan Tugas Mandiri)
- Resistor 220 Ohm (1 buah, atau 2 buah jika mengerjakan Tugas Mandiri)
- *Push Button / Tactile Switch* (1 buah)
- Resistor 10k Ohm (1 buah, untuk resistor *pull-down* tombol)
- Sensor Suhu & Kelembapan DHT11/22 (1 buah)

- Kabel *Jumper* (secukupnya)

#### 4.4.2 Instalasi Pustaka Khusus (*Manual via GitHub*)

Karena berbasis teknologi Asinkron pihak ketiga, Anda harus mengunduh kode sumber pustaka ini secara manual dalam format .zip dari GitHub, lalu memasangnya ke Arduino IDE via menu **Sketch > Include Library > Add .ZIP Library...**

Silakan kunjungi tautan berikut dan unduh berkas ZIP dari menu Code > Download ZIP:

1. **ESPAsyncWebServer:** <https://github.com/ESP32Async/ESPAsyncWebServer>
2. **ESPAsyncTCP:** <https://github.com/ESP32Async/ESPAsyncTCP>

(Perhatian: Jika Anda menggunakan mikrokontroler seri ESP32 di luar praktikum ini, pustaka TCP-nya bernama *AsyncTCP*).

### 4.5 Praktikum

#### 4.5.1 Praktikum 1: *Real-Time Dashboard & Sinkronisasi Tombol Fisik*

Skenario: Anda memiliki lampu LED (simulasi aktuator/lampu cerdas) yang bisa dihidupkan melalui sakelar fisik (*push button*) di *breadboard*, ATAU melalui tombol di antarmuka web *smartphone* Anda. Keduanya harus tersinkronisasi dua arah secara *real-time*. Jika Anda menekan tombol fisik di meja praktikum, status visual indikator di antarmuka *smartphone* harus otomatis berubah seketika, begitu pula sebaliknya jika dikontrol dari web.

##### Langkah Kerja Rangkaian:

1. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**. Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND NodeMCU** (*sama seperti rangkaian LED pada Modul 3*).
2. Hubungkan pin **DATA / OUT DHT** ke pin **D4 (GPIO 2)**. (VCC DHT ke 3V3, GND ke GND).
3. Pasang **Push Button** pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur **3V3 (3.3V)**. Hubungkan kaki seberang tombol ke pin **D2 (GPIO 4)** NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki **Resistor 10k Ohm**, lalu hubungkan ujung kaki resistor lainnya ke jalur **GND** (*konfigurasi resistor pull-down eksternal seperti pada Modul 1*).

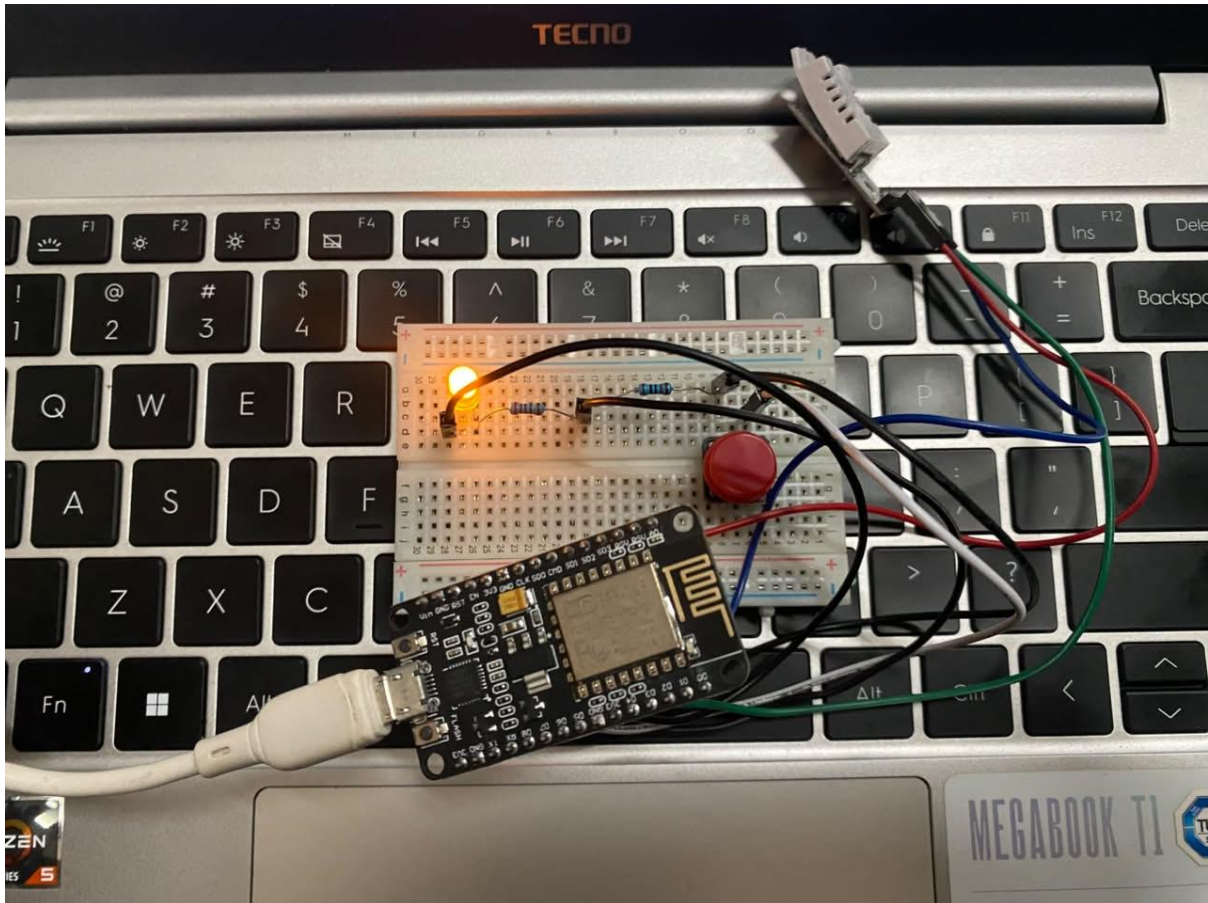
**Langkah Kerja Pemrograman:** Kode program ini cukup kompleks karena menggabungkan *Front-End* (HTML, CSS, JavaScript) dengan *Back-End* (C++). Agar *real-time response* tombol tidak terganggu oleh sensor DHT yang butuh jeda baca lama, kita menerapkan teknik pemrograman *non-blocking* menggunakan fungsi `millis()`.



Turn ON



Turn OFF



#### 4.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda terhadap arsitektur web modern!

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

**Jawab:** WebSockets lebih hemat karena koneksi antara ESP dan browser tetap terbuka, jadi data cuma dikirim saat ada perubahan. Sedangkan Meta Refresh harus terus-menerus memuat ulang halaman, jadi lebih banyak memakai internet.

Selain itu, WebSockets nggak bikin layar berkedip, karena yang berubah cuma datanya saja, bukan seluruh halaman web.

- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

**Jawab:** Kalau pakai `delay(3000)`, NodeMCU bakal **berhenti selama 3 detik**. Akibatnya WebSocket jadi **lemot merespons**. Misalnya tombol di browser ditekan, LED bisa telat menyala, dan data suhu juga telat diperbarui.

Intinya, `delay()` bikin NodeMCU **sibuk nunggu**, jadi komunikasi dengan browser nggak bisa berjalan lancar.

- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

**Jawab:** Yang jadi target perubahan teks ada 2 ID, yaitu:

1. `tempValue` untuk menampilkan nilai suhu.
2. `ledStatus` untuk menampilkan status LED ON/OFF.

Contohnya:

```
document.getElementById('tempValue').innerHTML = dataObj.suhu;
```

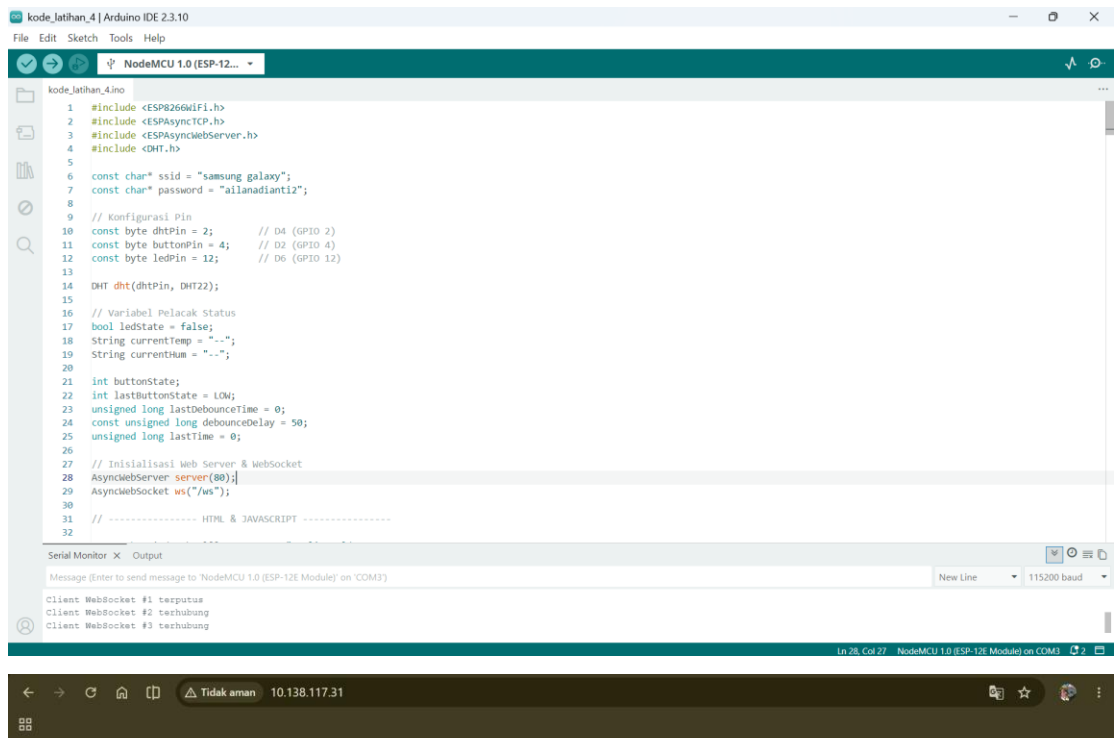
```
document.getElementById('ledStatus').innerHTML = "ON";
```

Jadi intinya itu, `tempValue` buat suhu dan `ledStatus` buat status LED.

- **Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

- i. **Sisi Back-End (C++):** Buat variabel penampung (misal: `String currentHum = "--"`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).
- ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`.

Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!



## Smart Room

Suhu: 32.30 Celcius

Kelembapan: 47.70 %

LED: OFF

Turn ON



## Smart Room

Suhu: 32.20 Celcius

Kelembapan: 46.90 %

LED: ON

Turn OFF

